서울시립대학교
UNIVERSITY OF SEOUL

전자전기컴퓨터설계실험II

실험 후 보고서

STM32와 FPGA의 SPI 송수신

학과	전자전기컴퓨터공학부
이름 / 학번	조성현 / 2023440129
작성일	2026년 11월 03일
대상	STM32 과제8 · STM32 과제9 · STM32 과제10 · STM32 과제11 · STM32 과제12
검증환경	Windows 11 / VS Code / STM32 CMake / VaporView / AMD Vivado 2026.1 / XSim / Serial Monitor

소스 식별

GitHub 계정: a40093632-ai

GitHub 저장소: https://github.com/a40093632-ai/ece2-lab7

제출 태그: lab7-post-v1

커밋 해시: 34e833820e680b2651ab4616191a4c672da12e40

1. 실험 목적

이번 실험에서는 SCK·MOSI·CS를 이용하여 STM32와 FPGA 사이에 한 바이트의 데이터를 전달하고, 송신 파형과 수신 결과를 비교한다. 과제8에서는 STM32가 생성한 SPI 파형을 오실로스코프로 관찰하고, 과제9에서는 두 MCU를 연결하여 수신 값을 확인한다. 과제10에서는 FPGA에 SPI 수신 회로를 구현하고, 과제11에서는 FPGA가 직접 SCK와 데이터를 생성하여 MCU로 송신한다. 과제12에서는 두 FPGA에 송신·수신 회로를 각각 적용하여 통신 결과를 확인한다.

FPGA 설계에서는 SCK 상승 에지 검출, 시프트 레지스터, 비트 카운터와 송신 상태 제어를 구현한다. 각 회로의 테스트벤치 결과를 파형과 비교하고, Vivado 합성·구현 결과 및 실제 보드의 UART 출력과 연결하여 살펴본다.

2. 관련 이론

비동기 입력의 동기화와 메타 안정성

외부 장치에서 들어오는 신호는 수신 회로의 내부 클록과 변화 시점이 일치하지 않을 수 있다. 이러한 신호가 플립플롭의 샘플링 시점 가까이에서 바뀌면 설정·유지 시간 조건을 위반하여, 출력이 정상적인 논리값으로 결정되는 데 시간이 걸리는 메타안정성이 발생할 수 있다.

대표적인 대응 방법은 수신 클록으로 동작하는 두 플립플롭을 직렬로 연결하는 것이다. 첫 번째 플립플롭의 출력을 다음 클록에서 다시 샘플링하여 메타안정성이 후단 회로로 전달될 가능성을 낮춘다. 다만 동기화는 위험을 줄이는 방법이며, 모든 상황에서 오류가 발생하지 않음을 보장하는 것은 아니다.[1]

이번 FPGA 수신 회로에서는 SCK·MOSI·CS를 각각 2단 레지스터로 받아들이고, 동기화된 SCK의 현재 값과 이전 값을 비교하여 상승 에지를 검출한다. 내부 클록은 50 MHz로 주기가 20 ns이며, SCK가 1 MHz일 때의 반주기는 500 ns이다. 이 시간 차이를 이용하여 외부 신호를 여러 내부 클록에 걸쳐 관찰한 뒤 비트를 수집한다. 따라서 입력 동기화뿐 아니라 MOSI의 안정 구간과 SCK·CS의 상대적인 순서도 함께 확인해야 한다.

3. 실험 환경 및 검증 방법

사용 장비와 연결

항목	설정 및 용도
STM32	NUCLEO-G474RE, SPI 마스터 송신 또는 슬레이브 수신
FPGA	Combo II-DLD S75, SPI 송신 및 수신 회로
FPGA 소자	xc7s75fgga484-1
오실로스코프	MCU 송신 클록과 데이터 파형 관찰
SPI 설정	Mode 0, 8비트, MSB 우선
PC 출력	STM32 VCP UART 또는 FPGA USB-UART
Serial Monitor	9600 baud, 8N1
FPGA 구현	Vivado 합성, 구현 및 비트스트림 생성
결과 관리	Git 및 Github

검증 순서

STM32는 코드 빌드 결과와 실제 UART 출력을 확인하였다. FPGA는 테스트벤치의 PASS 및 파형을 확인한 뒤, 개인 소스로 합성·구현과 비트스트림 생성을 진행하였다.

실습은 조원 코드로 진행하였고, 이후 개인 소스로 생성한 Vivado 결과와 비트스트림을 별도로 정리하였다.

실험별 설계 및 증빙 파일

항목	설계 및 증빙
MCU 송신	송신 코드
MCU 수신	수신 코드
FPGA 송신, 수신	RTL 소스
테스트벤치	시뮬레이션 파일
FPGA 핀 설정	XDC 파일
실험 증빙	사진, 파형, 구현 결과

4. STM32 과제8: MCU 송신 파형 확인

설계 및 예상 동작

STM32를 SPI 마스터로 사용하여 1초마다 한 바이트를 송신한다. 전송 전 CS를 LOW로 설정하고, 한 바이트를 보낸 뒤 HIGH로 해제한다. 오실로스코프에서는 한 바이트 전송 구간에 SCK 상승 에지가 8번 나타나야 한다. 또한 송신 값을 0x55로 고정하여 출력되는 비트열인 01010101에 맞는 파형이 관찰되어야 한다.

코드 동작

polling_routine()은 HAL_GetTick()으로 경과 시간을 확인하고, 1000 ms가 지나면 송신을 시작한다. CS를 LOW로 내린 뒤 HAL_SPI_Transmit()으로 한 바이트를 보내고, 종료 후 CS를 HIGH로 되돌린다. 송신 값과 함수 반환 값은 printf()로 출력한다.

오실로스코프 측정 및 송신 출력 결과

오실로스코프에 두 채널을 연결하여 송신 구간의 파형을 관찰하였다. 그림 1은 장비에서 파형을 측정한 사진이고, 그림 2는 측정 파형을 장비에서 저장한 화면이다.

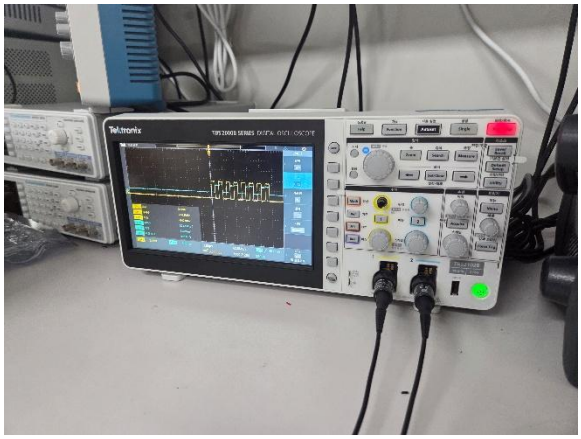


그림 1: 과제8 오실로스코프 시연 결과

그림 2에서 CH1에는 한 전송 구간에 8개의 클록 펄스가 나타난다. 클록은 전송 전후 LOW로 유지되며, 측정 주파수는 784.1 kHz이다. 예상 값과 비교하면 일치한 값으로 볼 수 있다. 또한 CH2에서도 전송 구간에 맞추어 신호가 변화한다. HIGH 구간이 4번 나타나는 신호로 MOSI의 0x55 데이터인 01010101이 출력된 파형임을 알 수 있다.

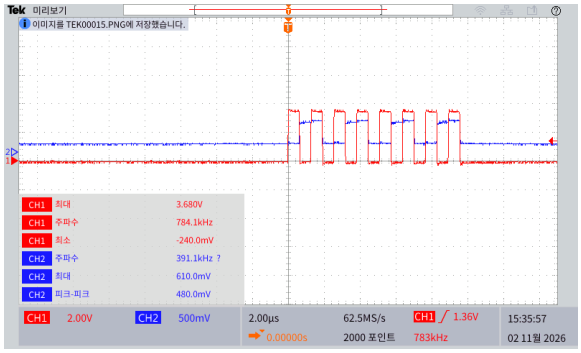


그림 2: 과제8 저장 파형과 SCK 주파수 측정값

Serial Monitor는 STM32의 VCP 포트를 9600 baud, Text 모드로 열어 확인하였다. 그림 3에서는 TX=55, result=0이 반복해서 나타난다. 이는 송신 값이 0x55로 설정되어 있고 SPI 송신 함수가 정상적으로 종료한 결과로 볼 수 있다.

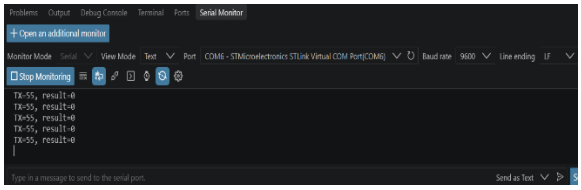


그림 3: 과제8 MCU 송신 값 및 함수 반환 값 출력

링크: [과제8 오실로스코프 및 송신 출력 자료]

5. STM32 과제9: MCU → MCU 수신 확인

설계 및 예상 동작

두 MCU를 각각 SPI 마스터와 슬레이브로 설정한다. 송신 MCU는 SCK·MOSI·CS를 출력하고, 수신 MCU는 선택된 구간에서 한 바이트를 받아 VCP UART로 출력한다. 송신값이 0x55라면 수신 화면에는 RX=55가 나타나야 한다.

코드 동작

송신 측은 과제8의 마스터 프로그램을 쓴다. 수신 측은 ece2_474_init()에서 SPI1을 해제한 뒤 SPI_MODE_SLAVE와 SPI_NSS_HARD_INPUT으로 다시 초기화한다. PA4는 AF5의 SPI1 NSS 입력으로 설정한다.

수신 함수는 HAL_SPI_Receive()로 한 바이트를 기다린다. 정상 수신하면 RX=%02X 형식으로 출력하고, 2000 ms 동안 데이터를 받지 못하면 타임아웃 메시지를 출력한다.

MCU 보드 시연 및 결과 비교

수신 MCU의 VCP UART를 Serial Monitor에서 확인하였다. 그림 4에는 RX=55가 반복해서 기록되어 있다. 이는 기본 송신 값 0x55에 대응하는 수신 결과이며, MCU 사이에서 한 바이트가 전달된 기록을 확인할 수 있다.

과제8이 송신 파형을 관찰하는 단계였다면, 과제9에서는 별도의 수신 MCU가 복원한 값을 확인하였다. 따라서 송신 프로그램의 출력뿐 아니라 수신 장치에서 확인한 값으로 통신 결과를 비교하였다.

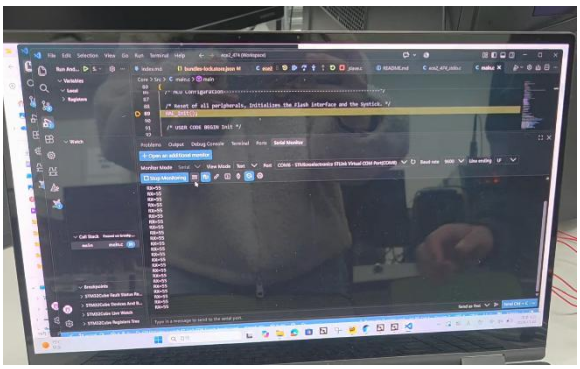


그림 4: 과제9 수신 MCU의 UART 출력

링크: [과제9 수신 출력 자료]

링크: [과제9 보드 시연 영상]

6. STM32 과제10: MCU → FPGA 수신 구현

설계 및 예상 동작

STM32를 마스터, FPGA를 슬레이브로 구성한다. FPGA는 CS가 LOW인 구간에서 SCK 상승 에지마다 MOSI를 읽고,

8비트가 모이면 수신값을 갱신한다. 설계에서는 수신 값을 LED에 반영하고 두 자리 16진수 문자와 줄 바꿈으로 변환하여 UART로 출력한다.

코드 및 회로 동작

spi_slave_rx는 외부 SCK·MOSI·CS를 각각 2단 레지스터로 동기화한다. 동기화된 SCK의 현재 값과 이전 값을 비교하여 상승 에지를 검출하고, 해당 시점의 MOSI를 시프트 레지스터에 저장한다.

여덟 번째 비트를 받으면 마지막 입력 비트까지 포함한 값을 rx_data에 기록하고 rx_valid를 한 내부 클록 동안 발생시킨다. 수신 도중 CS가 HIGH로 해제되면 시프트 레지스터와 비트 카운터를 초기화하여 미완성 데이터가 다음 전송에 섞이지 않게 한다.

Vivado 실험사진

```
run all
UART LINE 1 OK: SPI=55
UART LINE 2 OK: SPI=a5
UART LINE 3 OK: SPI=00
UART LINE 4 OK: SPI=ff
LAB7_SPI_RX_UART_PASS bytes=4 chars=16
```

그림 5: 과제10 SPI 수신 Vivado 시뮬레이션 PASS



그림 6: 과제10 SPI 수신 합성 및 구현 완료 결과

Summary

Resource	Utilization	Available	Utilization %
LUT	79	48000	0.16
FF	97	96000	0.10
IO	14	338	4.14

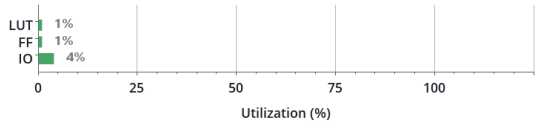


그림 7: 과제10 SPI 수신 FPGA 자원 사용량



그림 8: 과제10 SPI 수신 DRC 검사 결과

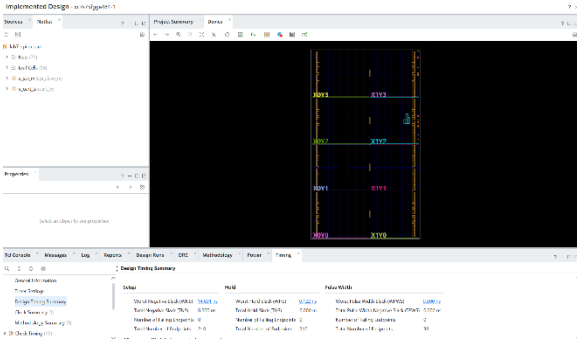


그림 9: 과제10 SPI 수신 타이밍 분석 결과

```
Algorithm : SHA256
Hash      : 85A80DF27833C8CCC16F55407CD97E65C92C55FC36828BBEACCA40BF1F8141A4
Path      : D:\Engineering\Vivado_Work\Lab7\lab7_spi_rx\project_1.runs\impl_1\lab7_spi_rx_uart.bit
```

그림 10: 과제10 SPI 수신 Bit Hash 확인

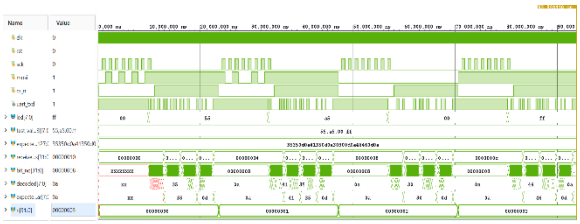


그림 11: 과제10 SPI 수신 및 UART 출력 파형

Vivado 시뮬레이션 및 파형 비교

그림 11에서는 각 CS LOW 구간에 SCK 펄스가 발생하고, 한 바이트의 전송이 끝난 뒤 LED 값이 00 → 55 → A5 → 00 → FF로 갱신된다. 처음의 00은 초기값이며, 이후 값은 순서대로 입력한 네 바이트에 대응한다.

UART 복원 데이터와 예상 문자도 함께 표시되어 있다. 0x55 입력은 문자 55, 0xA5는 A5, 0x00은 00, 0xFF는 FF로 출력되며, 각 값 뒤에 0D·0A가 이어진다. 이를 통해 SPI 수신 값이 LED 갱신과 UART 문자 출력으로 연결되는 순서를 확인하였다.

FPGA 보드 시연

그림 12는 두 보드의 연결과 LED 출력 상태를 보여준다. LED가 2, 4, 6번이 점등된 것은 송신 값 0x55의 이진 표현인 01010101이 표현된 결과로 볼 수 있다. 그리고 그림 13의 Serial Monitor 결과를 보면 TX=55, result=0이 반복해서 나타났다. 동시에 오른쪽 FPGA 측 UART 출력 값에서도 55가 반복해서 나타난 것을 보면 송신 값과 출력 값이 일치하는 것을 확인할 수 있다.

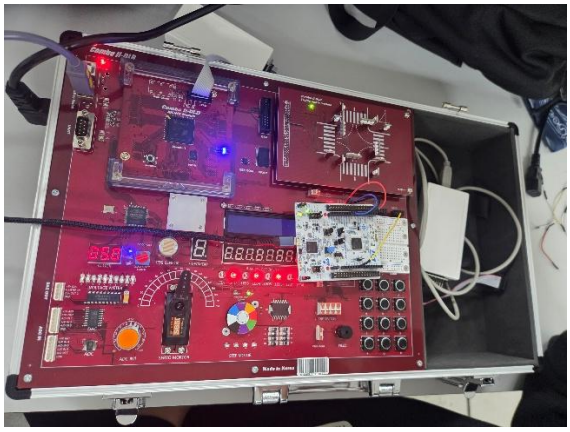


그림 12: 과제10 STM32-FPGA 연결 및 보드 시연

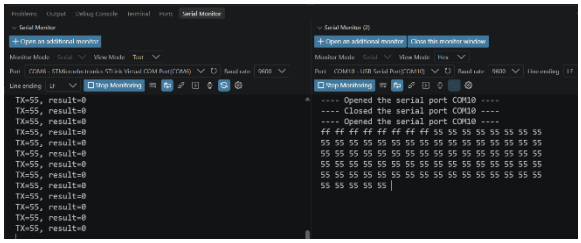


그림 13: 과제10 MCU 송신 및 FPGA UART 출력 기록

링크: [과제10 보드 증빙 자료]

7. STM32 과제11: FPGA → MCU 송신 구현

설계 및 예상 동작

FPGA를 SPI 마스터로 구성하고 MCU를 슬레이브로 설정한다. FPGA는 1 MHz 이하의 SCK와 MOSI·CS를 출력하고, 1초마다 한 바이트를 송신한다. 수신 결과는 MCU의 VCP UART에서 확인한다.

코드 및 회로 동작

spi_master_tx는 송신 요청을 받으면 데이터를 저장하고 CS를 LOW로 설정하면서 최상위 비트를 MOSI에 출력한다. 첫 상승 에지 전에 데이터가 준비되도록 하며, 이후 SCK 하강 에지에서 다음 비트로 변경한다.

마지막 비트의 전송이 끝나면 SCK를 LOW로 유지하고 CS를 일정 시간 더 유지한 뒤 해제한다. 이어서 busy를 내리고 done을 한 클록 동안 발생시킨다.

또한 FPGA 송신 설계에는 UART 출력은 없다. 실험에서 PC에 나타나는 수신 값은 MCU가 받은 SPI 데이터를 UART로 출력한 것이다.

Vivado 실험사진

```
run all
MASTER FRAME 1 OK: data=55 bits=8
MASTER FRAME 2 OK: data=55 bits=8
MASTER FRAME 3 OK: data=55 bits=8
LAB7_SPI_MASTER_PASS frames=3
```

그림 14: 과제11 SPI 송신 Vivado 시뮬레이션 PASS

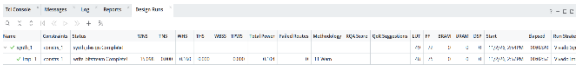


그림 15: 과제11 SPI 송신 합성 및 구현 완료 결과

Summary

Resource	Utilization	Available	Utilization %
LUT	48	48000	0.10
FF	75	96000	0.08
IO	13	338	3.85

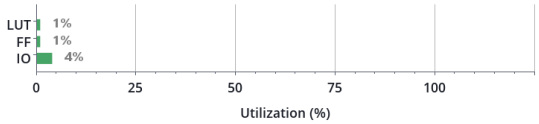


그림 16: 과제11 SPI 송신 FPGA 자원 사용량



그림 17: 과제11 SPI 송신 DRC 검사 결과

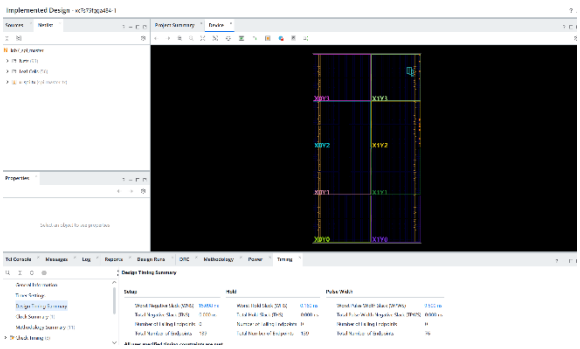


그림 18: 과제11 SPI 송신 타이밍 분석 결과

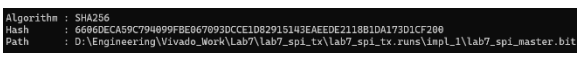


그림 19: 과제11 SPI 송신 Bit Hash 확인

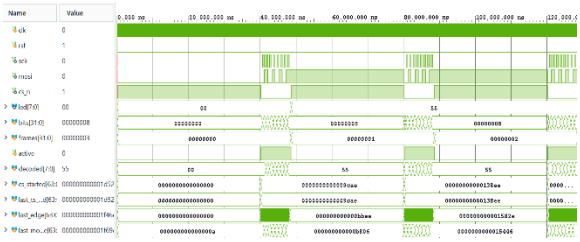


그림 20: 과제11 SPI 송신 vivado 파형

Vivado 시뮬레이션 및 파형 비교

그림 20에서는 CS가 LOW인 구간에 SCK 펄스가 발생하고 MOSI에 데이터가 출력된다. decoded는 전송 중 비트를 수집한 뒤 0x55로 완성되며, 프레임 수를 나타내는 frames는 전송 완료에 따라 증가한다. LED도 첫 전송 완료 후 0x55로 갱신된다.

FPGA 및 MCU 보드 시연

FPGA를 송신 측, STM32를 수신 측으로 사용하여 보드 시연을 진행하였다. 조원의 코드를 사용하여 송신 값은 0xAA로 고정되어 있다. 그림 21에서는 FPGA 보드의 LED가 1, 3, 5, 7번이 켜져 있고, 이는 0xAA의 이진수 표현인 10101010이 정확하게 표현된 값임을 알 수 있다. 또한, 그림 22의 STM32 가상 Serial Monitor 결과 값을 보면 수신 값을 확인할 수 있다. RX=AA가 반복해서 나타난 것을 보면 송신 값이 수신 값으로 정확하게 전달되었다고 볼 수 있다.

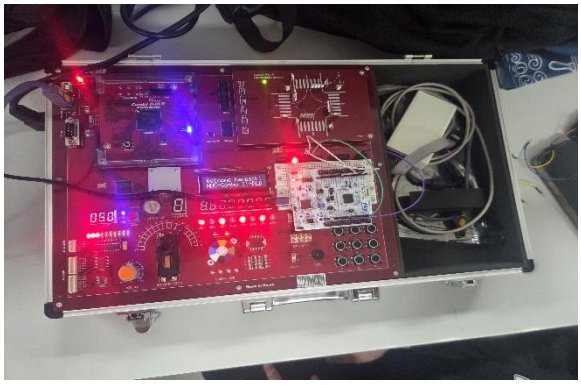


그림 21: 과제11 STM32-FPGA 연결 및 보드 시연

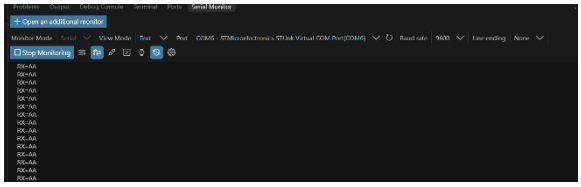


그림 22: 과제11 수신 MCU의 UART 출력

링크: [과제11 보드 증빙 자료]

8. STM32 과제12: FPGA → FPGA 송수신 구현

설계 및 예상 동작

FPGA 두 대를 각각 SPI 마스터와 슬레이브로 구성하여 데이터를 전달한다. 송신 FPGA는 SCK·MOSI·CS를 출력하고, 수신 FPGA는 CS가 LOW인 구간에서 SCK 상승 에지마다 MOSI를 읽어 한 바이트를 복원한다. 수신 결과는 FPGA의 LED와 PC Serial Monitor에서 확인한다.

회로 구성 및 검증 방법

송신 FPGA의 SCK·MOSI·CS를 수신 FPGA의 대응 입력에 연결하고 두 보드의 GND를 공통으로 연결하였다. 수신 측의 UART 출력은 보드의 USB Serial Port를 통해 PC에서 확인하였다. 이 과제의 데이터 전달에는 STM32를 사용하지 않는다.

FPGA 보드 시연

두 보드의 확장 단자 사이에 신호선을 연결하고, 한 쪽에서 SPI 신호를 출력하면 다른 쪽에서 이를 받도록 구현하였다. 그림 24를 보면 LED가 2, 4, 6, 8번이 켜져 있고, 이는 송신 값 0x55를 수신 받아 LED에 출력된 결과임을 보여준다. 그리고, 그림 25에서는 55가 줄 바꿈과 함께 반복해서 나타난다. 이를 통해 두 FPGA를 연결한 상태에서 수신 데이터가 LED와 PC 출력에 반영되는 결과를 관찰할 수 있다.

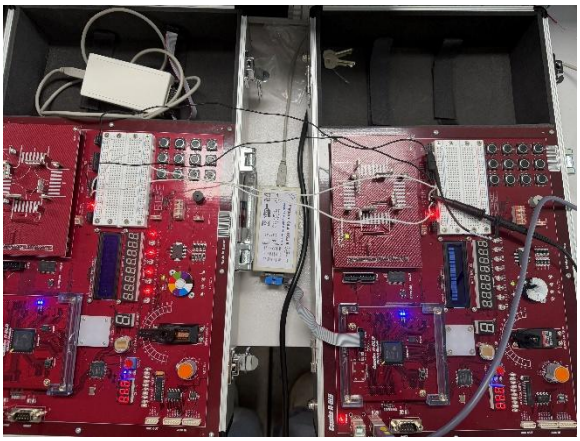


그림 23: 과제12 송수신 FPGA 연결 상태

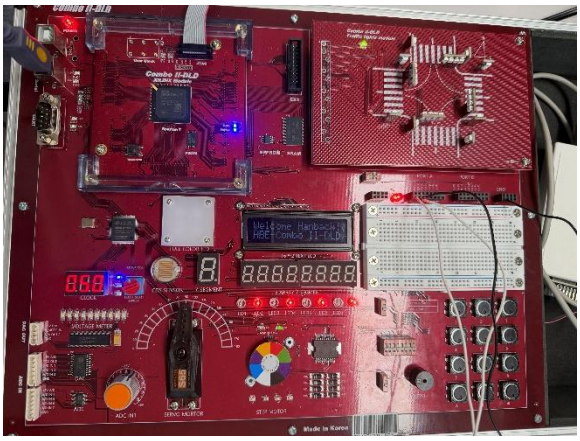


그림 24: 과제12 수신 FPGA의 배선 및 LED 표시

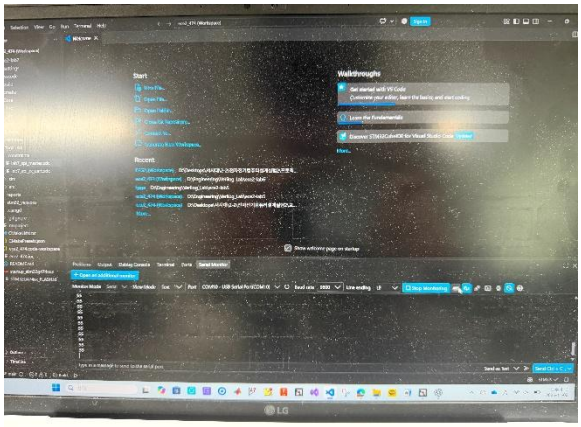


그림 25: 과제12 수신 FPGA의 Serial Monitor 출력

링크: 과제12 보드 시연 영상

링크: 과제12 보드 시연 사진

9. 문제 발생 및 해결 과정

UART 표시 방식에 따른 출력 해석

과제10과 과제12에서는 수신 결과를 Serial Monitor로 확인했지만, 저장한 화면의 표시 방식이 서로 달랐다. 과제10의 FPGA 출력은 Hex 모드이므로 화면의 55는 수신된 UART 바이트 0x55를 뜻한다. 과제12의 화면은 Text 모드이므로 문자 55가 표시된 결과이다.

두 화면에 같은 숫자가 나타나더라도 UART로 전달된 바이트 형식까지 같다고 판단할 수는 없다. 따라서 보고서에는 각 화면의 Text·Hex 설정을 함께 기록하고, SPI 수신값과 화면에 표시하는 형식을 구분하였다.

10. 예상 값 · Vivado · 보드 결과 비교

실험	예상 및 설계 검증	시연 결과
과제 8	SCK 약 781.25 kHz, 한 바이트 당 8펄스	8개의 클록 펄스 및 수신 펄스 관찰
과제 9	MCU 송신값을 다른 MCU에서 수신	수신 MCU에 RX=55 반복
과제 10	SPI 수신값을 LED와 UART에 반영	LED 점등, FPGA Hex 화면에서 초기 FF 이후 55 반복
과제 11	FPGA 송신 데이터를 MCU에서 수신	송신 보드 LED 점등, MCU에 RX=AA 반복
과제 12	두 FPGA 연결 후 수신값 표시	수신 LED 점등, FPGA Text 화면에 55 반복

11. Bitstream 해시 표

실험	Bitstream	SHA-256
수신	Lab7_spi_rx_uart.bit	85A80DF...
송신	Lab7_spi_master.bit	6606DEC...

링크: 비트스트림 파일

12. 결론

이번 실험에서는 STM32의 SPI 송신 파형을 측정하고, MCU와 FPGA의 송수신 역할을 바꾸어 데이터 전달 결과를 확인하였다. 오실로스코프에서는 한 바이트에 해당하는 8개의 클록 펄스가 나타났으며, SCK 주파수는 784.1 kHz로 측정되었다. MCU와 FPGA의 수신 출력도 확인하면서, 송신 함수의 정상 종료 기록뿐 아니라 데이터를 받은 장치의 값을 함께 살펴보아야 한다는 점을 이해하였다.

FPGA 수신 회로는 외부 입력을 동기화한 뒤 SCK 상승 에지마다 비트를 수집하고, 8비트가 모이면 수신값을 갱신한다. 송신 회로는 첫 상승 에지 전에 데이터를 준비하고 하강 에지에서 다음 비트로 변경한다. 이러한 동작을 파형과 연결하면서 클록과 데이터의 시간 관계가 통신에 중요하다는 것을 확인하였다.

과제12에서는 앞서 작성한 송신·수신 회로를 두 FPGA에 나누어 사용하였다. 수신 보드의 LED가 점등되고 Serial Monitor에 55가 반복되는 결과를 관찰하였다. 또한 UART의 Text·Hex 표시와 실제 데이터 값의 차이를 구분하고, 개인 설계의 구현 결과와 현장에 적용한 코드의 결과를 각각 정리하였다.

참고 문헌

[1] Ran Ginosar, "Metastability and Synchronizers: A Tutorial," *IEEE Design & Test of Computers*, vol. 28, no. 5, pp. 23–35, Sept.–Oct. 2011. DOI: 10.1109/MDT.2011.113.

https://webee.technion.ac.il/people/ran/papers/Metastability-and-Synchronizers.IEEEDToct2011.pdf?utm_source=chatgpt.com